SafeStay

Student Housing Management Platform

Technical Project Report

Comprehensive System Documentation

Architecture | Design Patterns | OOP Concepts | SOLID Principles

Research and Development Personnel:

- **Parthraj Singh Bhati — Student ID: 2401010320**
- **Piyush Sharma — Student ID: 2401010437**
- **Shourya Pratap — Student ID: 2401010445**
- **Anshuman Mehta — Student ID: 2401010082**
- **Vridhi Chaudhary — Student ID: 2401010336**

1. Executive Summary

SafeStay is a premium, full-stack student housing management and booking platform engineered to modernize and digitize the traditionally fragmented student accommodation market. By bridging the gap between students seeking housing, property owners managing listings, and administrators overseeing the ecosystem, SafeStay delivers a cohesive, real-time, and transparent experience for all stakeholders.

1.1 Platform Overview

The platform serves three distinct user roles, each with tailored dashboards and workflows:

- Students can search, filter, and book rooms in real time, view property details, manage bookings, and receive instant availability updates without manual refresh.
- Owners (Property Managers) can upload and manage property listings, monitor room statuses (Available, Booked, Under Maintenance), confirm or reject bookings, and access analytics dashboards.
- Administrators have system-wide oversight including user management, room inventory control, and complete booking visibility.

> **Mission Statement**
> SafeStay's mission is to eliminate the friction, opacity, and inefficiency of traditional student accommodation booking by delivering a secure, real-time, and feature-rich digital platform that empowers students to find safe housing and enables owners to manage properties with professional-grade tools.

1.2 Core Value Proposition

SafeStay addresses several critical pain points in student housing:

- Real-time synchronization prevents double-bookings through Optimistic Concurrency Control and live Socket.io events.
- Dynamic pricing engine allows flexible pricing based on room amenities and add-ons such as AC, Wi-Fi, and meal plans.
- Automated booking lifecycle management handles pending, locked, confirmed, and expired booking states without manual intervention.
- Role-based access control ensures each stakeholder sees only what is relevant to their role and responsibilities.

2. Technology Stack

SafeStay employs a modern, production-grade technology stack designed for scalability, type safety, and developer productivity. Every technology choice was deliberate, balancing performance, ecosystem maturity, and long-term maintainability.

Category	Technology	Purpose
Backend Runtime	Node.js + Express 5	High-performance async server with latest Express features
Language	TypeScript	End-to-end type safety across backend and frontend
ORM	Prisma ORM	Type-safe database client with migration support
Database	PostgreSQL	Relational database with ACID compliance
Frontend Framework	React 19	Component-based UI with latest concurrent features
Build Tool	Vite	Ultra-fast HMR and optimized production builds
Styling	Tailwind CSS v4	Utility-first styling with JIT compilation
Animations	Framer Motion	Declarative animation library for React
Icons	Lucide Icons	Consistent, open-source icon library
State Management	Zustand	Lightweight global state with minimal boilerplate
Server State	React Query	Caching, background sync, and async state management
Real-time	Socket.io	Bidirectional WebSocket communication
Validation	Zod	Runtime schema validation for requests and responses
Authentication	JWT + Cookies	Stateless auth with secure HttpOnly cookie storage

2.1 Backend Stack Deep Dive

The backend is built on Node.js with Express 5, leveraging TypeScript for complete type coverage. Prisma ORM abstracts raw SQL into a type-safe query interface, while PostgreSQL serves as the persistence layer with full ACID transactional support.

- Express 5 introduces async error handling improvements and router-level middleware enhancements that simplify route organization.
- Prisma generates a fully typed client from the schema, eliminating runtime type mismatches between the application and database layers.

- PostgreSQL's version field on the Room model enables Optimistic Concurrency Control — a critical mechanism to prevent race conditions during simultaneous booking attempts.

2.2 Frontend Stack Deep Dive

The frontend is a React 19 single-page application built with Vite, combining Tailwind CSS v4 for styling and Framer Motion for smooth UI transitions. State is split between Zustand for global client state and React Query for all server-synchronized data.

- React Query handles caching, background refetching, and stale-while-revalidate patterns, significantly reducing unnecessary API calls.
- Zustand provides a minimal, non-boilerplate-heavy alternative to Redux for sharing UI state across components.
- Framer Motion enables declarative page transitions and micro-interactions that enhance the user experience without sacrificing performance.

3. System Architecture & Design

SafeStay follows a modular, layered architecture that cleanly separates concerns across feature domains. The system is designed around the principle of high cohesion within modules and low coupling between them.

3.1 Backend Architecture

The backend is organized into feature modules, each encapsulating all related logic for a specific domain. This modular structure ensures that adding or modifying one feature domain does not impact unrelated modules.

Module	Responsibilities
Auth	User registration, login, logout, JWT issuance, cookie management
Booking	Room reservation lifecycle: pending → locked → confirmed → expired
Property	Room and property CRUD, owner management, status tracking
Maintenance	Ticket creation, assignment, status updates, room blocking
Room	Room search, filtering, availability querying, real-time sync

3.1.1 Layered Architecture Pattern

Each backend module follows a strict three-layer pattern:

- Controller Layer — Handles HTTP request parsing, response formatting, and route-level validation. Controllers are thin and delegate all business logic to services.

- Service Layer — Contains all business logic, orchestrates database operations, enforces rules, and manages cross-module interactions.
- Data Layer (Prisma) — Handles all database interactions. Services interact with Prisma models; no raw SQL is used in application code.

3.2 Frontend Architecture

The frontend follows a component-based architecture with distinct organizational layers that separate UI rendering from data fetching and business logic.

- Pages — Top-level route components that compose smaller components and orchestrate data fetching via React Query hooks.
- Components — Reusable, stateless or locally stateful UI elements with clear prop interfaces.
- Hooks — Custom React hooks (e.g., useRoomSync) that encapsulate Socket.io subscriptions, React Query calls, and derived state computations.
- Services — Axios-based API client modules that abstract all HTTP communication, providing typed request and response functions.

3.3 Database Design

The database schema is designed in PostgreSQL with Prisma, structured around five core entities that reflect the real-world relationships within the student housing domain.

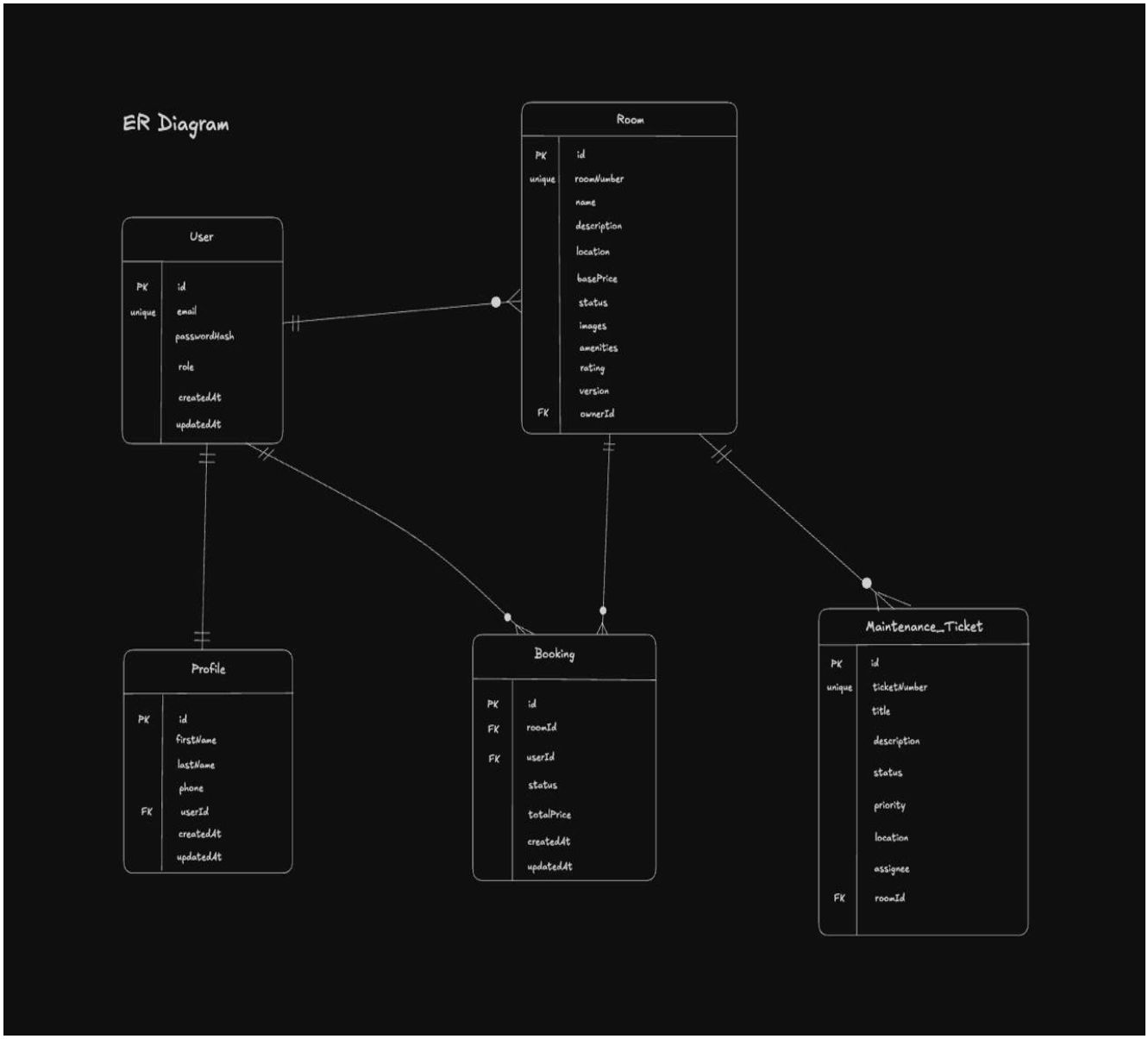

Figure 1: Entity-Relationship (ER) Diagram — SafeStay Database Schema

3.3.1 Key Schema Decisions

Optimistic Concurrency Control (OCC)
The Room entity includes a version integer field. Every booking update increments this version. If two simultaneous booking attempts read the same version and attempt to write, only the first commit succeeds — the second fails with a version mismatch, preventing double-bookings without pessimistic row-level locking.

Entity	Key Fields
User	id, email (unique), passwordHash, role, createdAt, updatedAt
Profile	id, firstName, lastName, phone, userId (FK → User)
Room	id, roomNumber, name, basePrice, status, amenities, rating, version, ownerId (FK → User)
Booking	id, roomId (FK → Room), userId (FK → User), status, totalPrice, createdAt, updatedAt
Maintenance_Ticket	id, ticketNumber (unique), title, status, priority, location, assignee, roomId (FK → Room)

4. System Use Cases & Actor Interactions

The following use case diagram illustrates the complete interaction surface of the SafeStay system across its three primary actors: Students, Owners, and Administrators. Extended use cases (shown with dashed arrows) represent conditional or optional behaviors.

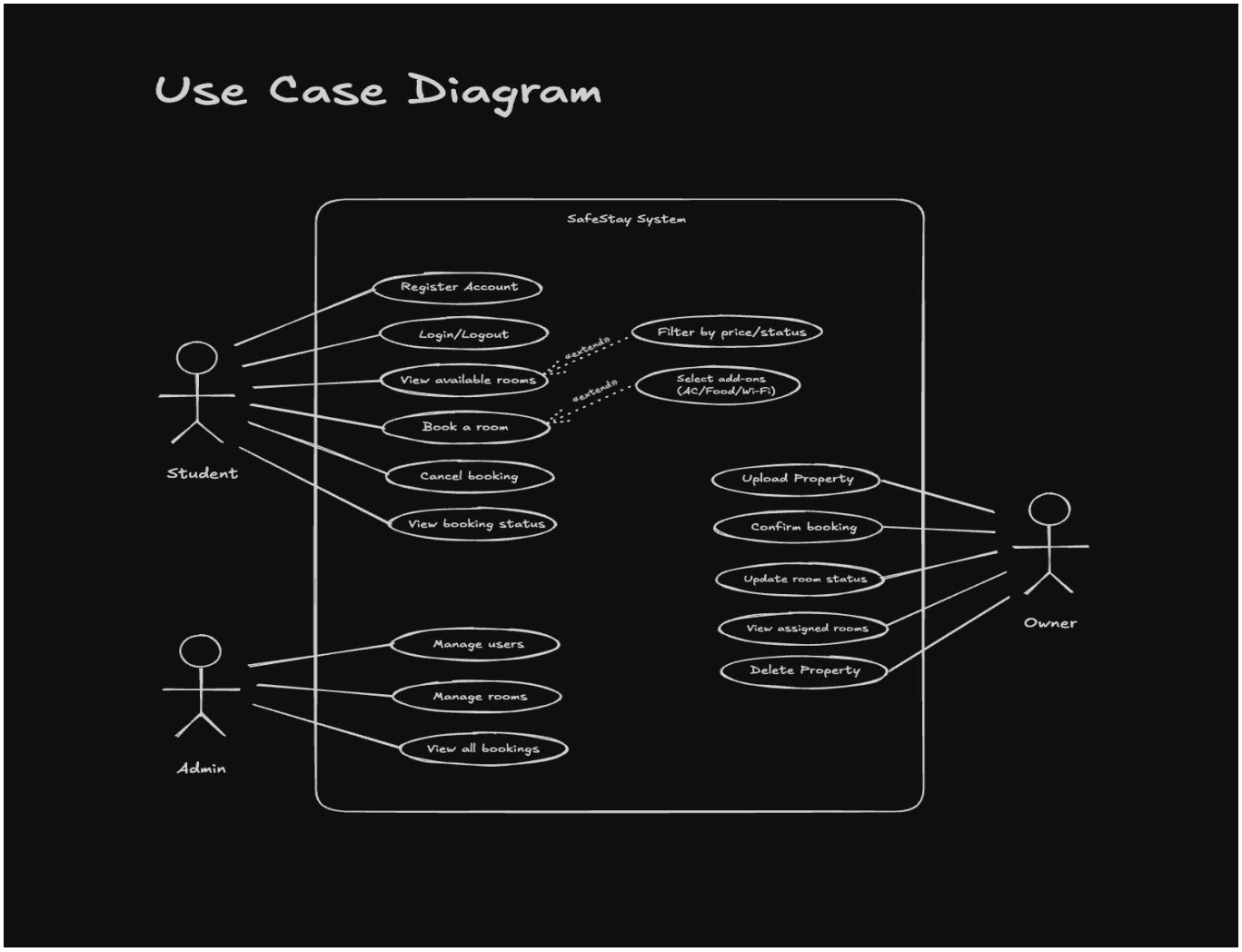

Figure 2: Use Case Diagram — SafeStay System Actor Interactions

4.1 Student Actor

- Register Account / Login / Logout — Standard authentication lifecycle.
- View Available Rooms — Browse the room catalog with real-time availability; extends to Filter by price/status for advanced searching.
- Book a Room — Initiate the booking flow; extends to Select Add-ons (AC, Food, Wi-Fi) for dynamic pricing.

- Cancel Booking / View Booking Status — Manage active and historical reservations.

4.2 Owner Actor

- Upload Property — Create new room listings with details, images, and pricing.
- Confirm Booking — Accept or reject pending booking requests from students.
- Update Room Status — Mark rooms as Available, Booked, or Under Maintenance.
- View Assigned Rooms / Delete Property — Manage the full property portfolio.

4.3 Admin Actor

- Manage Users — View, suspend, or modify user accounts across all roles.
- Manage Rooms — System-wide room inventory oversight and status control.
- View All Bookings — Complete booking ledger with filtering and reporting.

5. Object-Oriented Programming Concepts

SafeStay is architected with a deep application of OOP principles, ensuring that real-world domain concepts (rooms, bookings, pricing, maintenance) are modeled as coherent, reusable, and maintainable software entities.

5.1 Encapsulation

Business logic and data are encapsulated within well-defined service classes, preventing unauthorized direct access and ensuring that changes to implementation details do not leak across module boundaries.

> **Example: BookingService**
> BookingService encapsulates all booking state transitions, validation rules (e.g., checking room availability before confirming), price calculation orchestration, and database commit logic. External consumers call public methods like createBooking() or cancelBooking() without knowing the internal sequence of operations.

- BookingService — Encapsulates the complete booking lifecycle from initiation to expiry.
- PricingEngine — Contains all pricing calculation logic; external code never manipulates price fields directly.
- MaintenanceService — Encapsulates ticket creation, assignment, and status transitions.

5.2 Abstraction

Complexity is hidden behind clean, simple interfaces. High-level components interact with abstractions rather than implementations, enabling internal complexity to grow without affecting consumers.

> **Example: PricingEngine Abstraction**
> The BookingController calls pricingEngine.calculate(room, addOns) and receives a totalPrice. It has no awareness of whether the calculation uses a decorator chain, strategy pattern, or any other mechanism. This abstraction boundary means the pricing logic can be completely replaced without touching the controller.

5.3 Inheritance & Polymorphism

The State Pattern implementation (detailed in Section 7) uses polymorphism extensively — all booking state classes implement a common BookingState interface, allowing the BookingContext to call transition methods uniformly regardless of the current concrete state.

- PendingState, LockedState, and ConfirmedState all implement BookingState and can be substituted interchangeably — this is polymorphism in direct practice.
- Decorator pattern classes in the pricing module share a common interface, allowing the PricingEngine to treat base prices and decorated prices identically.

5.4 Composition Over Inheritance

SafeStay preferentially uses composition to build complex behaviors from simple, reusable parts rather than deep inheritance hierarchies that create tight coupling.

- BookingContext composes a BookingState object rather than inheriting from a base booking class.
- PricingEngine composes Decorator objects (WiFiDecorator, ACDecorator, MealDecorator) to build complex pricing calculations from simple units.

6. SOLID Design Principles

The SOLID principles are systematically applied throughout SafeStay's codebase, ensuring the system remains maintainable, extensible, and testable as it grows in complexity.

Letter	Principle	Core Idea
S	Single Responsibility	Every class has exactly one reason to change
O	Open/Closed	Open for extension, closed for modification
L	Liskov Substitution	Subtypes are substitutable for base types

Letter	Principle	Core Idea
I	Interface Segregation	Clients depend only on methods they use
D	Dependency Inversion	Depend on abstractions, not concretions

6.1 S — Single Responsibility Principle (SRP)

Every class in SafeStay has a single, well-defined responsibility. No class is responsible for more than one concern.

- Controllers — Exclusively handle HTTP request/response lifecycle. No business logic lives in controllers.
- Services — Exclusively handle business rules and data orchestration. No HTTP-specific code exists in services.
- State Classes — Each state class (PendingState, LockedState, ConfirmedState) manages only the transitions and behavior valid for that specific booking state.
- Middleware — Each middleware function (authGuard, roleGuard, zodValidate) has a single cross-cutting concern.

6.2 O — Open/Closed Principle (OCP)

The system is open for extension but closed for modification. New capabilities can be added without altering existing, tested code.

> **Pricing Extension Example**
> Adding a new 'Parking' add-on requires only creating a new ParkingDecorator class that wraps the base price with the parking surcharge. The PricingEngine, all existing decorators, and all tests remain untouched.

- New booking states (e.g., 'CheckedIn') can be added by implementing BookingState — no switch/case blocks need modification.
- New pricing add-ons are new Decorator classes — the engine's composition logic doesn't change.
- New API endpoints are new route files and controllers — the existing routing infrastructure is extended, not modified.

6.3 L — Liskov Substitution Principle (LSP)

All concrete state implementations can be used interchangeably through the BookingState interface. The BookingContext never needs to check which concrete state it holds — it simply calls the interface method and trusts the correct behavior is delivered.

- PendingState.confirmBooking() transitions to LockedState.
- LockedState.confirmBooking() transitions to ConfirmedState.
- ConfirmedState.confirmBooking() throws an appropriate 'already confirmed' error.

All three conform to the BookingState contract — the substitution is seamless.

6.4 I — Interface Segregation Principle (ISP)

SafeStay uses specific, narrow interfaces and Data Transfer Objects (DTOs) so that consuming classes depend only on the subset of the interface they actually need.

- CreateBookingDTO contains only the fields required for booking creation — no unnecessary fields from the full Booking model are included.
- RoomAvailabilityQuery contains only filter fields relevant to searching — not the full Room schema.
- Service interfaces expose only the methods relevant to their consumers, avoiding god-object interfaces.

6.5 D — Dependency Inversion Principle (DIP)

High-level modules (Controllers) depend on abstractions (Service interfaces), not on concrete database implementations. This decoupling enables service implementations to be swapped or mocked without affecting the controllers.

- BookingController depends on IBookingService, not on BookingService directly.
- BookingService depends on the Prisma abstraction layer, not on raw PostgreSQL queries.
- This chain of abstraction means unit tests can inject mock services into controllers without spinning up a database.

7. Design Patterns

SafeStay implements five well-established Gang of Four design patterns, each chosen to solve a specific architectural challenge in a clean, idiomatic way.

7.1 State Pattern — Booking Lifecycle Management

Location: Backend/src/modules/booking/flow/

The State Pattern manages the complex lifecycle of a booking. Rather than using brittle if/else or switch/case blocks that check a status string, the behavior at each stage is delegated to dedicated state objects.

State Class	Responsibility
PendingState	Initial state after a booking request. Can transition to Locked or Cancelled.
LockedState	Room is temporarily reserved while payment is processed. Can transition to Confirmed or Expired.

State Class	Responsibility
ConfirmedState	Booking is complete and room is secured. Terminal successful state.
ExpiredState	Booking expired due to timeout. Terminal failure state.

> **Why State Pattern?**
> Without the State Pattern, adding a new booking stage (e.g., 'CheckedIn') would require finding and modifying every switch/case block across the codebase. With the State Pattern, it is simply a new class that implements BookingState — all existing state classes and the BookingContext remain untouched.

7.2 Decorator Pattern — Dynamic Pricing Engine

Location: Backend/src/modules/booking/pricing/

The Decorator Pattern enables flexible, composable pricing calculations. A base room price is wrapped with additional cost decorators for each amenity or add-on the student selects.

- BaseRoomPrice — The starting price from the Room model.
- WiFiDecorator — Wraps base price and adds the Wi-Fi surcharge.
- ACDecorator — Wraps any price and adds the AC surcharge.
- MealDecorator — Wraps any price and adds the meal plan cost.

These decorators can be combined in any order and combination, yielding accurate total prices for any amenity configuration without combinatorial explosion of pricing methods.

7.3 Observer Pattern — Real-Time Room Locking

Implementation: Socket.io event system

When a student initiates a booking, the server emits a room:locked event to all connected clients. All client-side observers (useRoomSync hooks) receive this event and immediately update their local UI state, marking the room as unavailable without requiring a page refresh.

- Server emits: room:locked, room:available, room:status_changed events.
- Client observes: useRoomSync hook subscribes to these events and invalidates React Query cache, triggering UI updates.
- Benefit: All users viewing the same room listing see real-time availability changes simultaneously.

7.4 Singleton Pattern — Database Client

Location: Backend/src/config/database.ts

The Prisma client is instantiated once as a module-level singleton. All service classes import and share this single instance, ensuring connection pool efficiency and preventing multiple database connections from being opened unnecessarily.

> **Singleton Rationale**
> Prisma maintains an internal connection pool. Creating multiple PrismaClient instances would open multiple pools, exhausting database connections. The Singleton Pattern guarantees exactly one pool exists for the application's lifetime.

7.5 Strategy Pattern — Pricing Calculation

The PricingEngine acts as a Strategy Selector. Depending on the add-ons and room type provided, it selects and chains the appropriate decorator strategy to produce the final price. This separates the 'how to calculate' from the 'when and what to calculate', enabling different pricing strategies to be applied without changing the engine's public interface.

8. Key Features & Modules

8.1 Advanced Booking System

The booking module is the core of SafeStay, implementing a sophisticated multi-state reservation lifecycle with real-time locking to prevent double-bookings.

- Real-Time Reservations — Socket.io events lock rooms the moment a student starts the booking flow, preventing concurrent conflicts.
- Dynamic Pricing Engine — Decorator-based price assembly supports flexible amenity combinations with accurate total cost calculation.
- ExpiryService — An automated background service that monitors pending and locked bookings, expiring them after a configurable timeout and releasing the room back to available status.
- State Machine — The PendingState → LockedState → ConfirmedState flow ensures every booking transition is valid and auditable.

8.2 Property & Room Management

Owners access a dedicated dashboard for complete property lifecycle management:

- Room listing creation with images, descriptions, amenities, base pricing, and location details.
- Real-time room status management (Available / Booked / Maintenance) synchronized across all connected clients via useRoomSync.
- Booking confirmation workflow where owners review pending requests and accept or reject them.
- Analytics dashboard showing occupancy rates, booking history, and revenue summaries.

8.3 Maintenance Ticketing System

The Maintenance module provides a structured workflow for handling room repairs without disrupting the booking system:

- Ticket creation with title, description, priority (Low / Medium / High), and location details.
- Unique ticket numbers (ticketNumber field) enable easy tracking and reference.
- Assignee field links tickets to specific maintenance personnel.
- Room status is automatically set to Maintenance when an active ticket blocks the room, preventing new bookings.

8.4 Role-Based Access Control (RBAC)

SafeStay enforces strict role separation through JWT-decoded role claims verified by server-side middleware:

- Students — Access to room search, booking flow, and personal booking history only.
- Owners — Access to property management, booking confirmation, and maintenance ticketing for their properties.
- Administrators — System-wide access to all user accounts, rooms, and bookings.

9. Security & Performance

9.1 Security Architecture

- JWT Authentication — Tokens are signed with a server-side secret and stored in HttpOnly cookies, preventing XSS attacks from accessing auth credentials.
- Zod Schema Validation — All incoming request bodies are validated against strict Zod schemas at the middleware layer before reaching controllers, preventing malformed data from reaching business logic.
- Role Guards — Middleware verifies JWT role claims on every protected route, ensuring students cannot access owner endpoints and vice versa.
- Optimistic Concurrency Control — The Room version field prevents race conditions during simultaneous booking attempts at the database level.
- Password Hashing — User passwords are hashed with bcrypt before storage; plaintext passwords never persist.

9.2 Performance Optimizations

- React Query Caching — Room listings and booking data are cached client-side with configurable stale times, reducing redundant API calls during normal navigation.
- Prisma Query Optimization — Only required fields are selected in database queries (Prisma select), avoiding over-fetching of large JSON fields like images and amenities.

- Singleton Database Client — A single Prisma connection pool serves all requests, maximizing PostgreSQL connection efficiency.
- Vite Build Optimization — Tree-shaking, code splitting, and asset optimization are applied at build time, minimizing JavaScript bundle sizes for fast initial page loads.
- Socket.io Event Scoping — Room events are emitted to specific rooms (socket.io rooms) rather than broadcasting to all connected clients, reducing unnecessary network traffic.

10. System Design Principles

Beyond SOLID and OOP, SafeStay adheres to broader system design principles that ensure long-term reliability, observability, and maintainability.

10.1 Separation of Concerns (SoC)

Every layer of the system has a distinct, non-overlapping responsibility. The HTTP layer knows nothing about pricing. The pricing layer knows nothing about database schemas. The database layer knows nothing about business rules.

10.2 DRY — Don't Repeat Yourself

Common logic is extracted into shared utilities and middleware. JWT verification, Zod validation, and error formatting are each defined once and reused across all modules.

10.3 Fail Fast & Validate Early

Zod validation middleware rejects malformed requests at the network boundary, before business logic executes. This prevents invalid data from propagating through the system and ensures error messages are generated at the earliest possible point.

10.4 Event-Driven Architecture (Partial)

The real-time features of SafeStay employ an event-driven pattern via Socket.io. State changes (room locked, booking confirmed) trigger events that propagate to interested consumers without tight coupling — producers do not know which clients are listening.

10.5 Stateless API Design

The REST API is stateless — each request contains all information needed to process it (via JWT in cookie). No server-side session state is maintained, enabling horizontal scaling of the backend without session affinity requirements.

11. Development Status & Future Roadmap

11.1 Current Progress

SafeStay has completed its foundational milestone with the following capabilities fully operational:

- PostgreSQL database migration complete with Prisma schema and all five entity models.
- Real-time room synchronization integrated via Socket.io with useRoomSync hook.
- JWT authentication with role-based middleware fully implemented.
- Booking lifecycle state machine (Pending → Locked → Confirmed → Expired) operational.
- Decorator-based dynamic pricing engine functional with AC, Wi-Fi, and Meal add-ons.
- Owner and Student dashboards with distinct workflows deployed.
- Maintenance ticketing system with priority management active.

11.2 Future Roadmap

Feature	Description
Payment Gateway	Integration with Stripe or Razorpay for in-app payment processing and automated receipts.
AI Room Recommendations	ML-based recommendation engine suggesting rooms based on student preferences, budget, and past bookings.
Mobile Application	React Native port of the student-facing interface for iOS and Android platforms.
Persistent Notifications	Email and push notification system for booking confirmations, cancellations, and maintenance updates.
Advanced Analytics	Owner revenue dashboards with occupancy heatmaps, seasonal pricing insights, and comparative analytics.
Multi-language Support	Internationalization (i18n) support for non-English speaking student populations.

12. Conclusion

SafeStay represents a well-architected, full-stack solution to the real-world problem of student housing management. Its technical depth is evidenced by the deliberate application of OOP

principles, SOLID design guidelines, and proven Gang of Four design patterns across a modern TypeScript codebase.

The State Pattern brings clarity to complex booking workflows. The Decorator Pattern enables flexible pricing without code bloat. The Observer Pattern via Socket.io delivers the real-time experience that modern users expect. The Singleton Pattern ensures resource-efficient database access. Together, these patterns form a codebase that is not only functional today but designed to scale and evolve with minimal technical debt.

> **Technical Excellence Summary**
> SafeStay achieves a rare combination of pedagogical clarity and production readiness — demonstrating OOP, SOLID, and system design principles in concrete, working implementations rather than abstract theory. It serves equally well as a portfolio showcase, a learning reference, and a foundation for a commercial product.

Appendix A: System Diagrams

A.1 Entity-Relationship Diagram

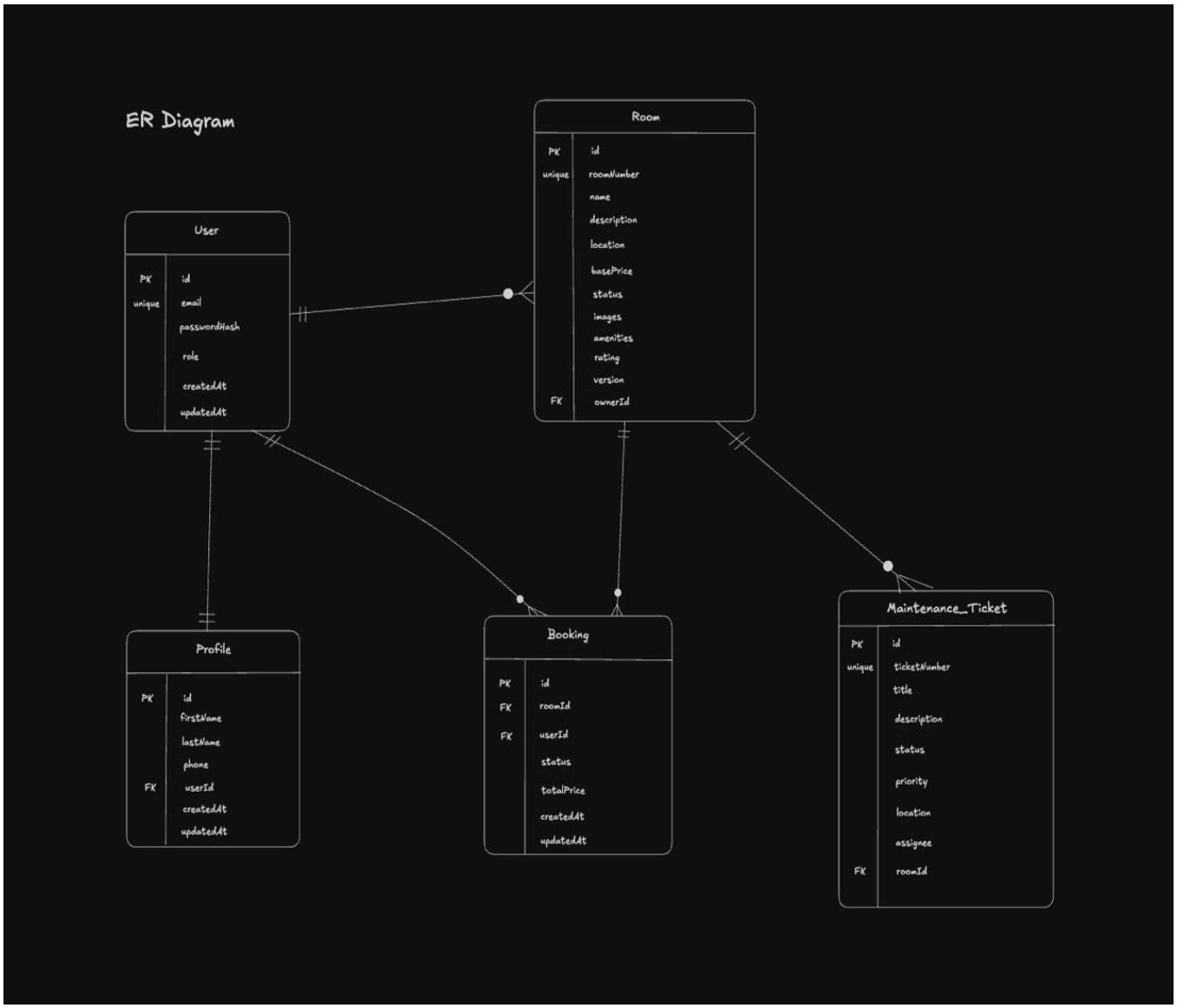

Figure A.1: Complete ER Diagram — SafeStay Database Schema

A.2 Use Case Diagram

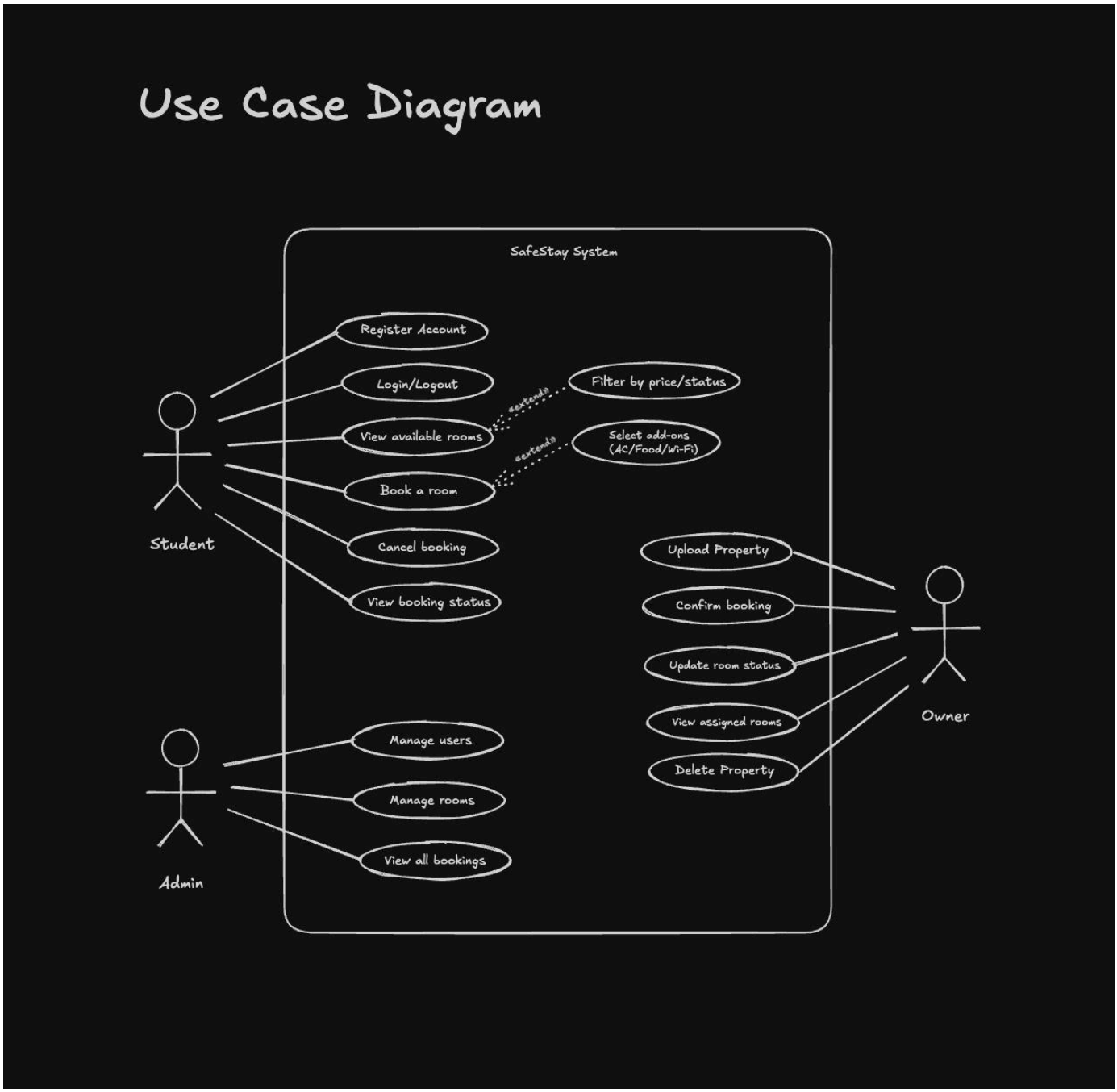

Figure A.2: Use Case Diagram — Actor Interactions